

Sistemi operativi: introduzione e concetti di base

Antonio Pecchia

antonio.pecchia@unisannio.it

Dipartimento di Ingegneria
Laurea in Ingegneria Informatica
Sistemi Operativi



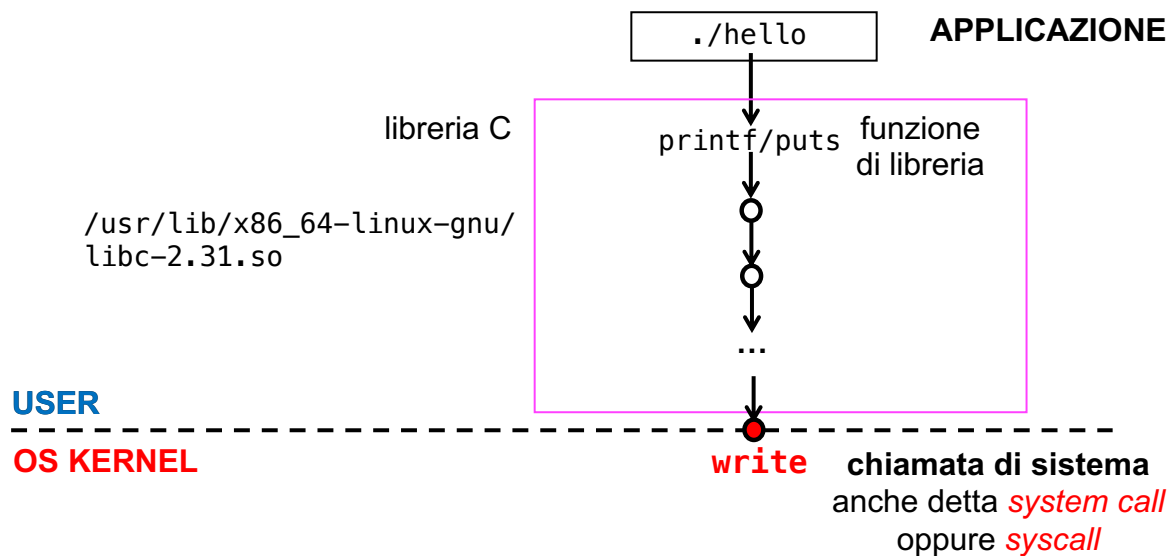
UNIVERSITÀ DEGLI STUDI
DEL SANNIO Benevento

Informazioni generali

- ❑ Metodi didattici:
 - lezioni frontali ed esercitazioni d'aula;
- ❑ Testo e materiale:
 - *I moderni sistemi operativi*, Tanenbaum, Bos. 5a edizione, Pearson (2023);
 - presentazioni delle lezioni;
 - materiale esercitativo, di approfondimento e risorse disponibili in rete.



Warm-up task



Warm-up task

- Alcuni puntatori agli esempi appena visti:
 - chiamate di sistema disponibili:
 - `find /usr/include -name unistd_64.h`
 - <https://gpages.juszkiewicz.com.pl/syscalls-table/syscalls.html>
 - **more on** `syscall`: `man syscall`
 - utility `ldd`:
 - elenca gli *shared object* richiesti da un programma;
 - <https://man7.org/linux/man-pages/man1/ldd.1.html>
 - utility `strace` (useful options: `-k -p --trace=...`):
 - traccia **chiamate di sistema** e **segnali** Linux;
 - <https://man7.org/linux/man-pages/man1/strace.1.html>

INTRODUZIONE

Cap. 1: Introduzione – par. 1.1

A first view

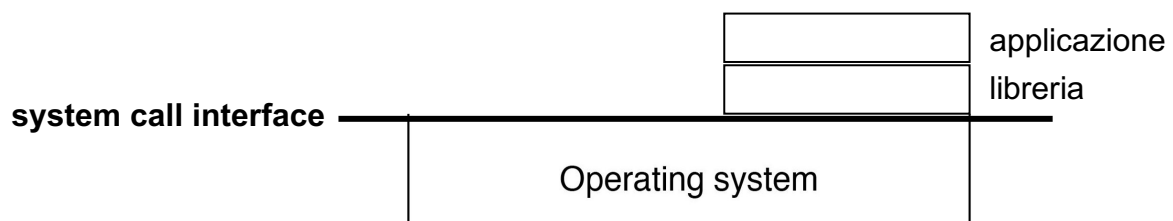


Figure 1-1. Where the operating system fits in.

A first view

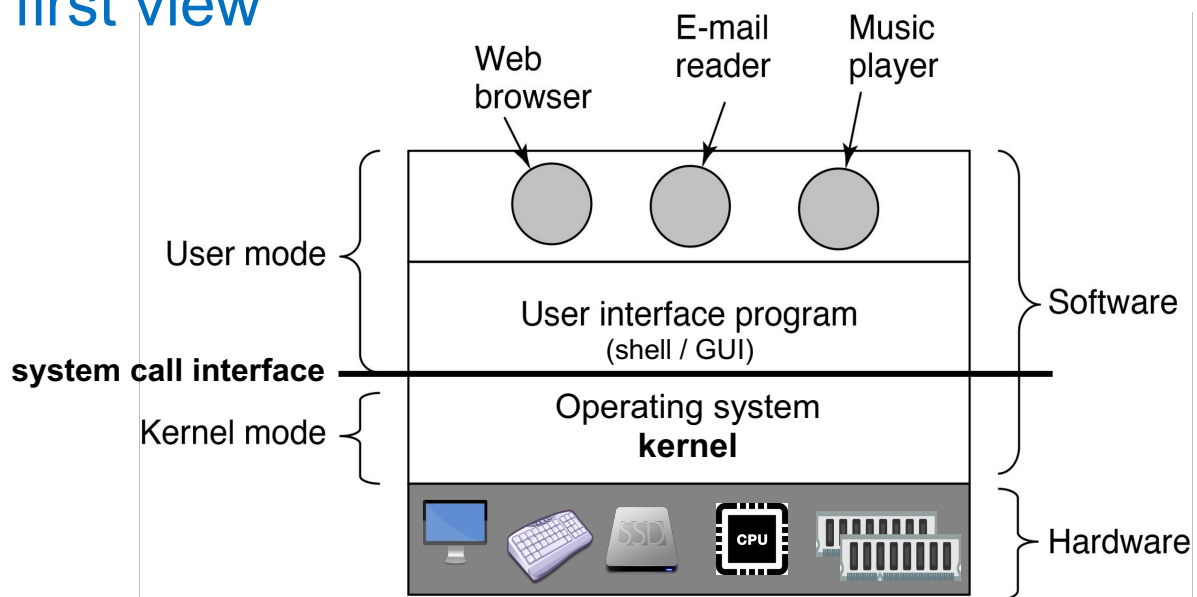


Figure 1-1. Where the operating system fits in.

Kernel e sistema operativo

- **Kernel** (chiamato anche **nucleo**):
 - **software** "mediatore" tra applicazioni e HW;
 - parte del SO che risiede in memoria centrale;
 - esegue in risposta* a:
 - **system calls**, **eccezioni**, **interruzioni**.
- **Introduzione par 1.1 del testo:** *"É difficile definire che cosa sia un S.O. al di là della semplice affermazione che si tratta di software che gira in modalità kernel..."*

* <https://w3.cs.jmu.edu/kirkpams/OpenCSF/Books/csf/html/KernelMechanics.html>

(Sez. 2.3.3)

Kernel e sistema operativo

- **Kernel** (chiamato anche **nucleo**):
 - **software** "mediatore" tra applicazioni e HW;
 - parte del SO che risiede in memoria centrale;
 - esegue in risposta* a:
 - **system calls**, **eccezioni**, **interruzioni**.
- **Sistema operativo** (in senso più ampio):
 - **kernel** + **sistema di base** (consente ad un SO di avviarsi e di presentare una interfaccia all'utente), **programmi di utilità** (task manager, deframmentazione), **librerie di sistema**.

* <https://w3.cs.jmu.edu/kirkpams/OpenCSF/Books/csf/html/KernelMechanics.html>

(Sez. 2.3.3)

Alcune precisazioni

- **Dal cap. 1 (Fig. 1.1) del testo:** *il programma con cui interagiamo shell (testo) e GUI (icone) "non è in realtà parte del sistema operativo":*
 - in altri testi-presentazioni si tende a ritenerli parte.
- I **programmi applicativi** (compilatori, ambienti grafici, programmi di ufficio, browser, client email, ...) **non sono** sistema operativo.

One more representation

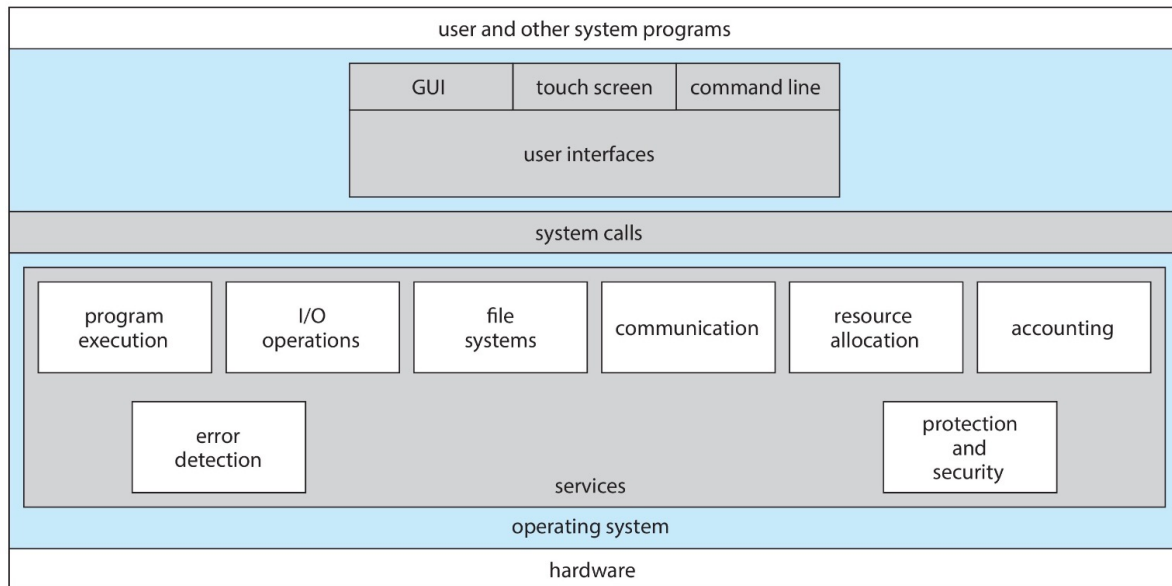
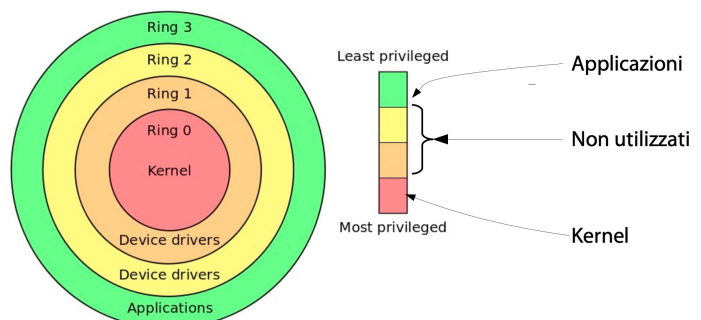
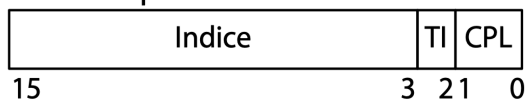


Figure from Silberschatz et al. 2018

Kernel mode vs user mode

- Nelle CPU Intel* sono previsti 4 livelli di privilegio (*ring*):
 - valore su 2 bit chiamato **current privilege level (CPL)** – lowest 2 bits of the *code segment (CS)* register;
 - in pratica, sono usati 2 livelli: **0 (kernel mode)** e **3 (user mode)**.

Selettore per codice



*in generale, in una CPU c'è un registro (oppure eventuali flag), che indica il *mode* (user o kernel) in cui si trova la CPU.

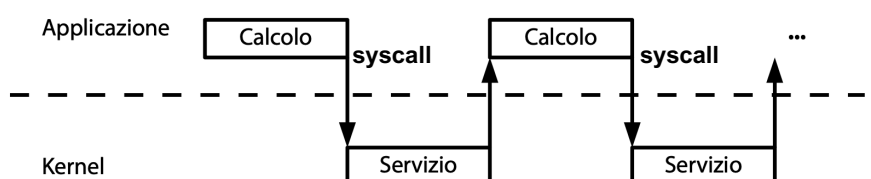
Kernel mode vs user mode

- ❑ **Kernel (o supervisor) mode** (è permesso tutto):
 - accesso consentito a tutto l'hardware e possibilità di eseguire **qualsiasi** istruzione della macchina;
 - operazioni di I/O (in/out), abilitare-disabilitare interrupt, etc;
 - **istruzioni privilegiate** (possono essere usate solo in kernel mode):
cli, sti, iret, hlt, ...
- ❑ **User mode** (privilegi ridotti):
 - disponibile solo un sottoinsieme di istruzioni macchina (per es., mov, load, add, sub, and, jmp, ...);
 - per es., **non può**: eseguire istruzioni assembly per l'I/O e, quindi, aprire file, stampare a video → **bisogna invocare il kernel!**

Kernel mode vs user mode

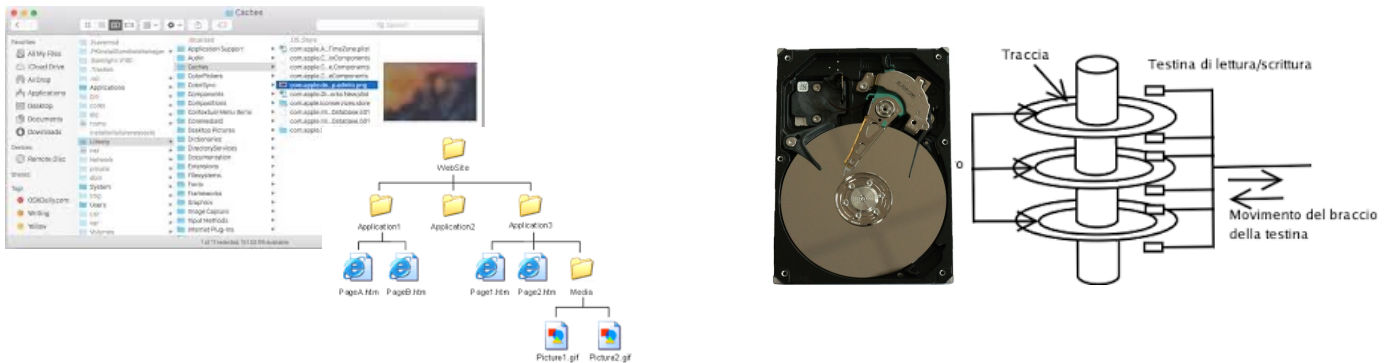
- ❑ In un certo istante, siamo o in **user** o in **kernel mode**:
 - la CPU deve essere in kernel mode per poter eseguire codice del kernel;
 - l'unico modo per un programma user mode di cambiare il livello di privilegio è tramite una **system call**.
- ❑ **System call**: unico meccanismo con cui una applicazione può richiedere un servizio al kernel.

i programmi in user mode richiedono l'esecuzione di operazioni privilegiate al **kernel** tramite system call



‘Cosa fa’ un sistema operativo

- Fornisce un insieme di **astrazioni**:
 - interfacce tramite cui è possibile usare la rete, disco, I/O, etc., **in maniera semplificata**;
 - per es. **file** invece di **byte memorizzati in un disco**;



‘Cosa fa’ un sistema operativo

- Fornisce un insieme di **astrazioni**:
 - interfacce tramite cui è possibile usare la rete, disco, I/O, etc., **in maniera semplificata**;
 - per es. **file** invece di **byte memorizzati in un disco**;
- Esempi di astrazione:
 - file / file system → memoria secondaria;
 - spazio di indirizzamento → memoria centrale;
 - processi / thread → esecuzione programmi;
 - socket → rete.

‘Cosa fa’ un sistema operativo

- ❑ **Gestore** delle risorse:
 - gestione delle risorse hardware e dei criteri con cui assegnare una risorsa;
 - controllo della condivisione (**multiplexing**): nel **tempo** (per es. CPU) e nello **spazio** (per es., la memoria).

una notizia: anche il sistema operativo stesso usa risorse e le deve condividere con i programmi applicativi

Principali funzioni di un sistema operativo

- ❑ User interface (UI): per es., command line, GUI;
- ❑ Program execution: caricamento in memoria, esecuzione;
- ❑ Operazioni I/O;
- ❑ File-system e file;
- ❑ Comunicazione (stesso computer o tramite rete);
- ❑ Rilevamento errori;
- ❑ Allocazione delle risorse: per es., cicli CPU, memoria centrale, I/O;
- ❑ Protezione e sicurezza.

CENNI STORICI

Cap. 1: par. 1.2 e 1.4

Images source: Wikipedia (<https://en.wikipedia.org/>)

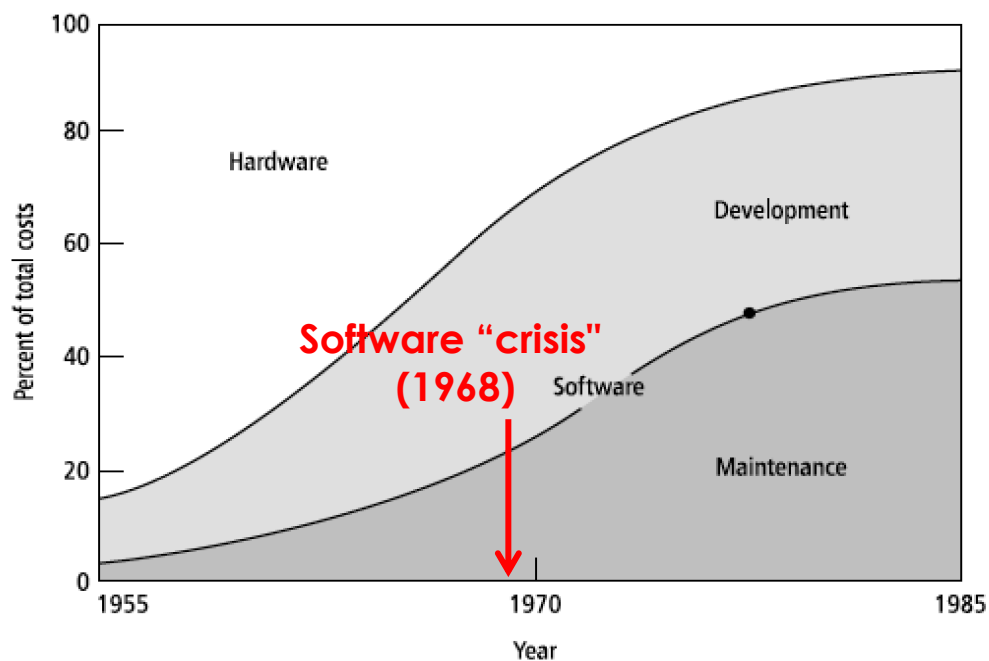
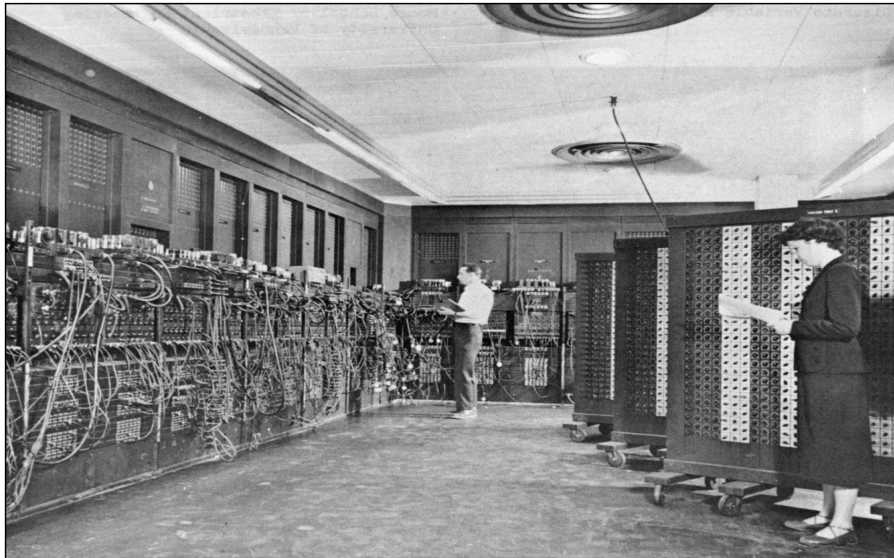


Fig. from *Software Engineering: Principle and Practice*, Hans van Vliet, Wiley (2007)

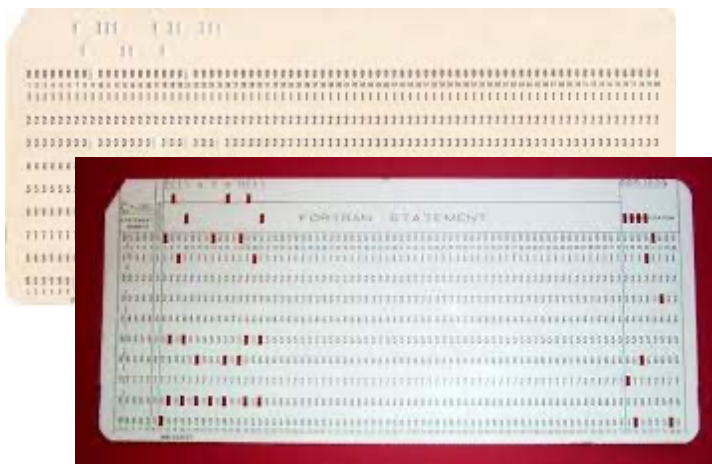
Prima generazione (metà '40 - metà '50)



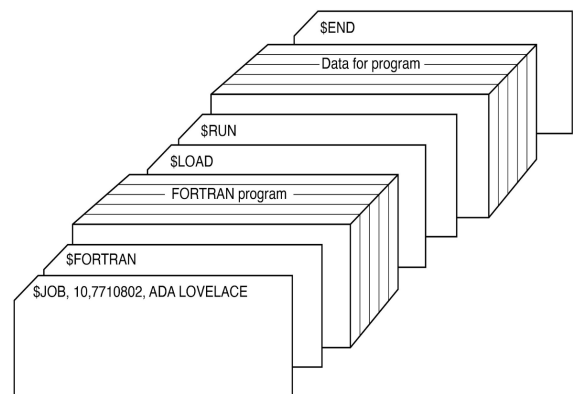
ENIAC – Electronic Numerical Integrator and Computer (1946), 17468 valvole termoioniche, circa 3000 istruzioni / secondo

Seconda generazione (metà '50 - metà '60)

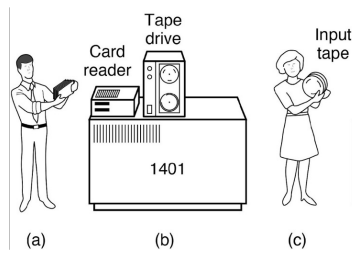
- Transistor – sistemi batch;



Struttura di un job

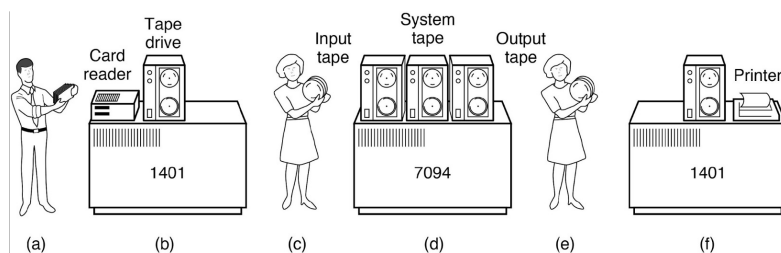
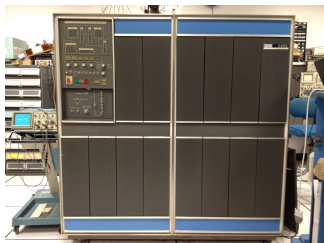


Seconda generazione (metà '50 - metà '60)



Seconda generazione (metà '50 - metà '60)

- OS: Fortran monitor system (FMS), IBSYS (OS IBM)



Terza generazione (metà '60 – '80)

- Circuiti integrati.

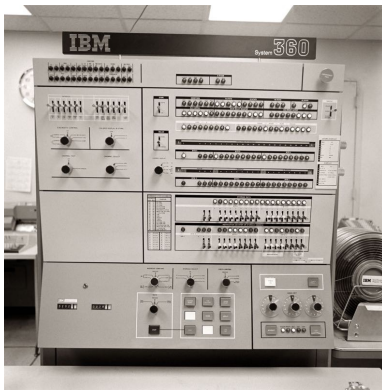


IBM 2260
(1964)



DEC VT05
(1970)

IBM 5150
(1981)

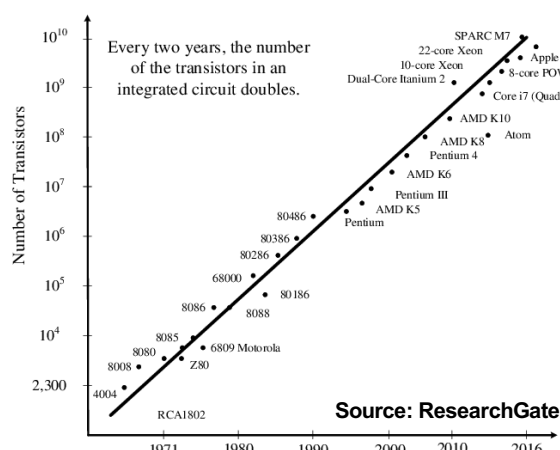


IBM 360 (1965)

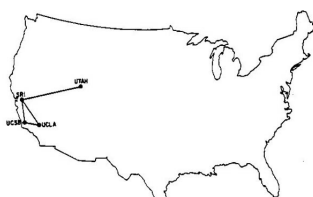


DEC PDP-11 (1970)

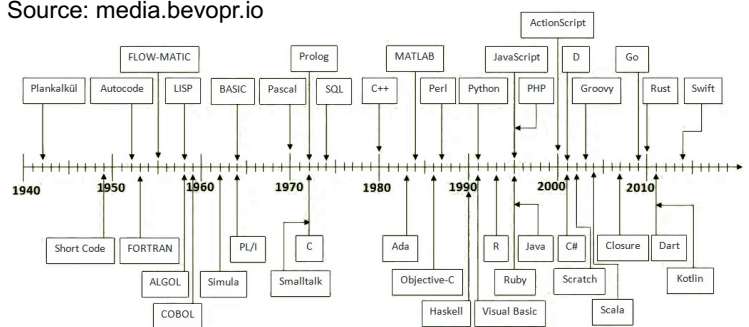
Sidenote



ARPANET
(1969)



Source: media.bevopr.io



HD IBM 305
Disk Storage Unit
5M (1956)



Apple M2 Pro (12-core 64-bit ARM64 SoC, SIMD , caches)	40,000,000,000 ^[190]	2023
Apple M2 Max (12-core 64-bit ARM64 SoC, SIMD , caches)	67,000,000,000 ^[190]	2023
Apple M2 Ultra (two M2 Max dies)	134,000,000,000 ^[191]	2023
AMD Epyc Bergamo (4th gen/97X4 series) 9-chip module (up to 128 cores and 256 MB (L3) + 128 MB (L2) cache)	82,000,000,000 ^[192]	2023
AMD Instinct MI300A (multi-chip module, 24 cores, 128 GB GPU memory + 256 MB (LLC/L3) cache)	146,000,000,000 ^{[193][194]}	2023

Sidenote: LOC* stimate per alcuni OS

- ❑ [Microsoft Windows](#): 50 million lines of code (2018);
- ❑ [Linux](#) kernel version 5.10 (2020): circa 27.8 million lines of code;
- ❑ [macOS 10.15](#) Catalina 86 million lines of code stimate;
- ❑ [Android](#) mobile OS (version 10): circa 12 million lines of code.

50 righe x 1000 pagine → 50.000 righe

*LOC = lines of code (una metrica software)

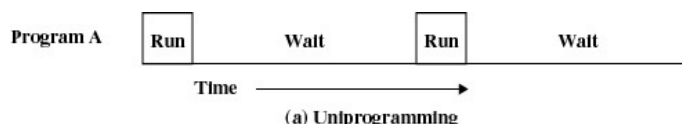
Aspetti relativi ai SO

- Problemi di sviluppo e idee fondamentali.
- Esempio **IBM OS/360**:
 - software complesso, milioni di righe di codice (cresciute da 1 a **10 milioni**), **migliaia** di bug;
 - si stima abbia richiesto **5000 anni uomo di sviluppo** tra il 1963-1966 (*nota: l'ingegneria del software nasce nel 1968*);
 - picchi di 1000 persone (programmatore, progettisti, personale tecnico, ...) a fronte di un team iniziale di 60 programmatori;
 - **4 volte il costo stimato iniziale**.

"adding manpower to a software project that is behind schedule delays it even longer"

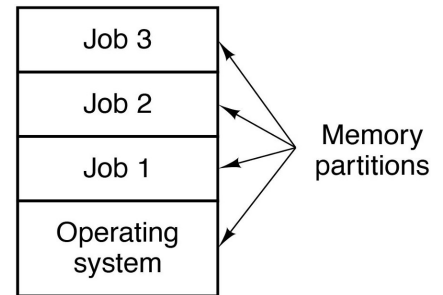
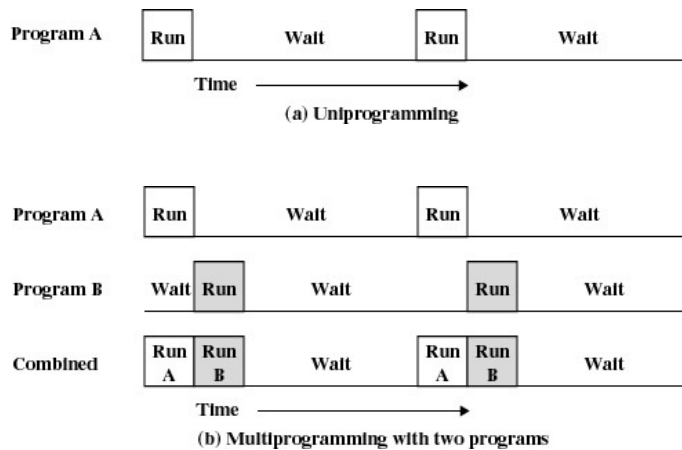
Monoprogrammazione vs Multiprogrammazione

- In un sistema **monoprogrammato** la CPU si pone in stato "inattivo" finché l'I/O termina.



Monoprogrammazione vs Multiprogrammazione

- **Multiprogrammazione: più job** caricati in memoria:
 - mentre un job attende il completamento dell'I/O, un altro job può usufruire della CPU.



Monoprogrammazione vs Multiprogrammazione

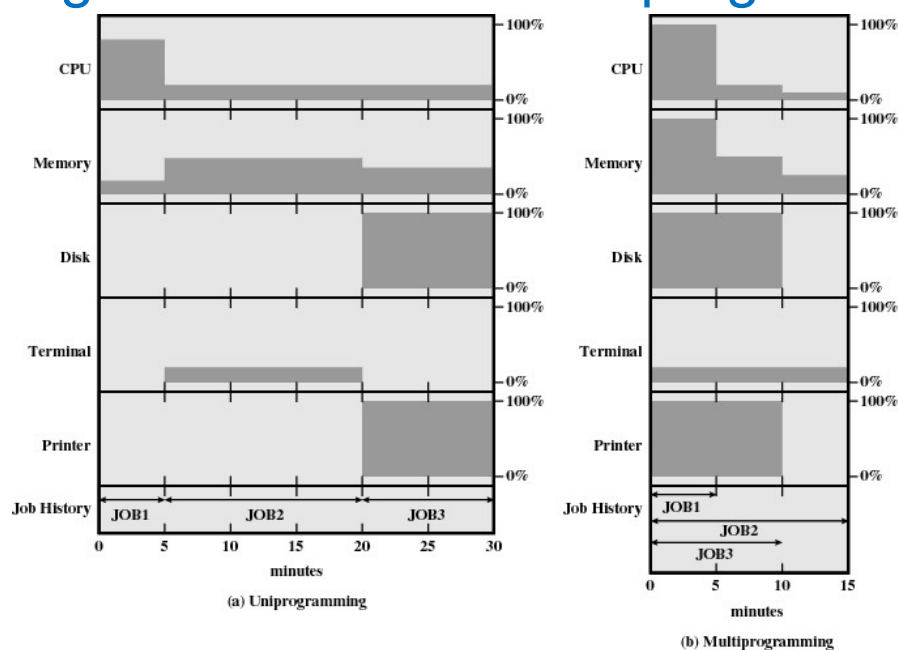
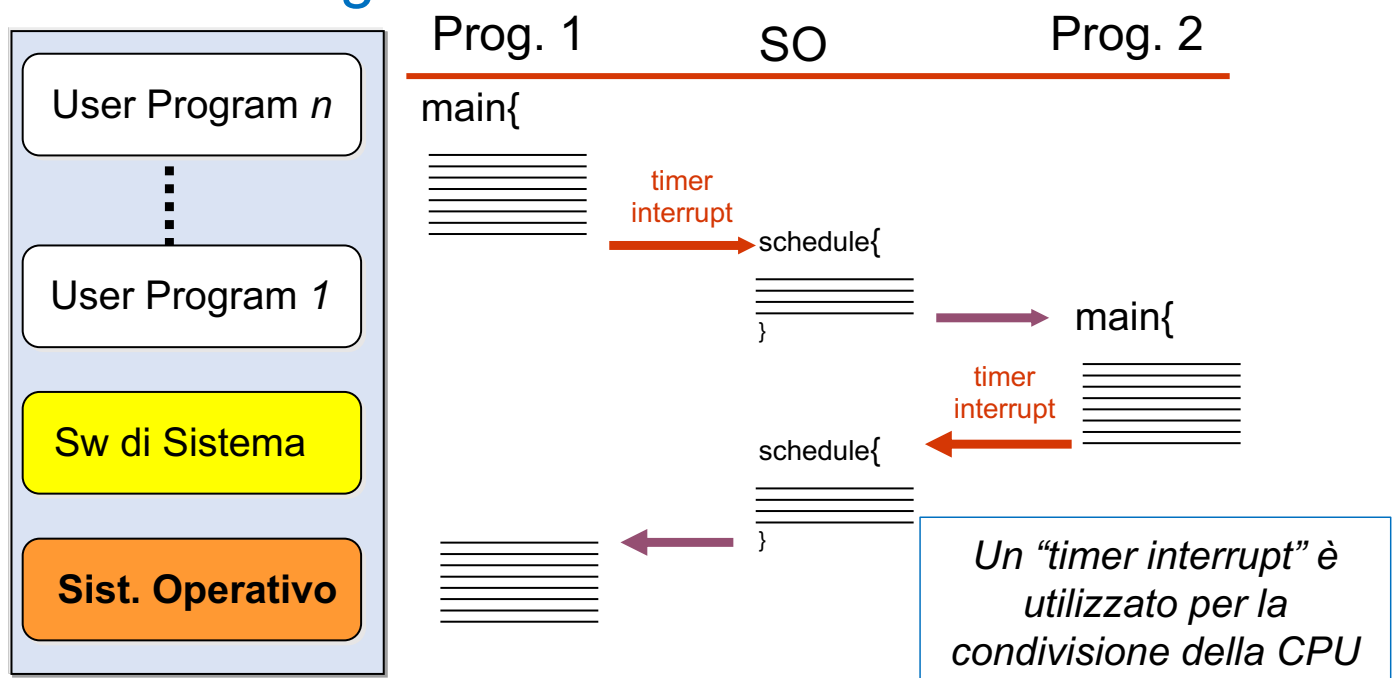


Figure 2.6 Utilization Histograms

Timesharing (multitasking)

- Una variante della multiprogrammazione:
 - ciascun job possiede, **a turno**, la CPU per un intervallo di tempo denominato **quanto**.
- **Motivazioni:**
 - gestione di un gruppo di task interattivi;
 - più utenti accedono simultaneamente al sistema da più terminali;
 - il tempo del processore è condiviso tra più utenti.

Timesharing



Timer interrupt frequency

The timer interrupt's frequency is another configuration worth mentioning to understand the [low-latency Ubuntu Linux kernel](#).

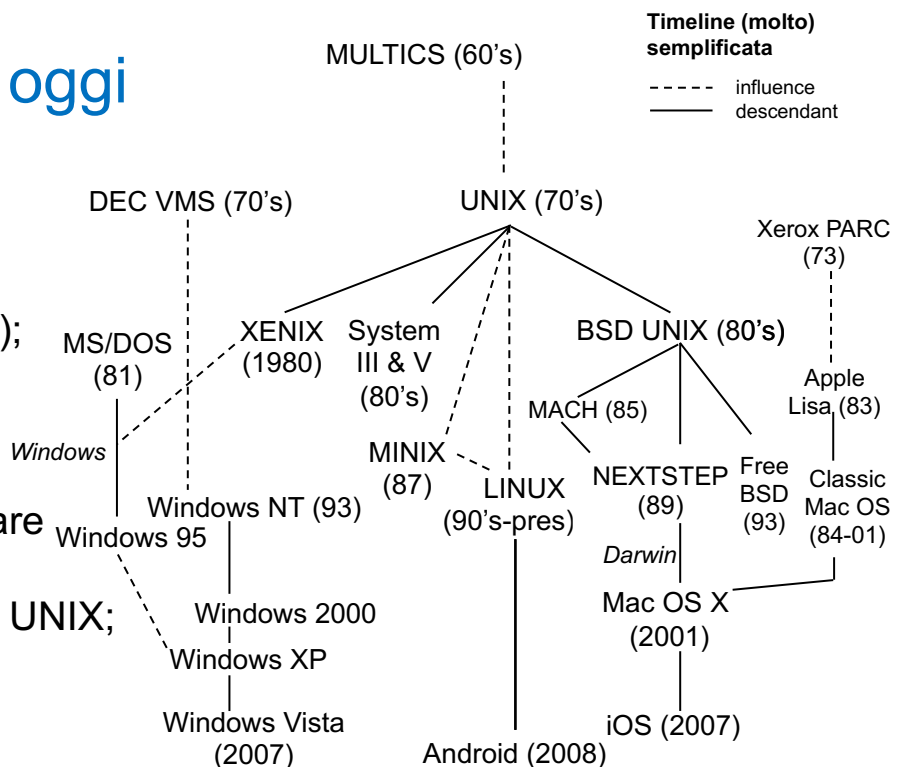
The timer interrupt handler interrupts the kernel at a rate set by the **HZ** constant. The frequency affects the timer resolutions as a 100 Hz value for the timer granularity will yield a max resolution of 10ms (1 Hz equating to 1000ms), 250Hz will result in 4ms, and 1000Hz in the best-case resolution of 1ms [4].

<https://ubuntu.com/blog/industrial-embedded-systems-ii>

Dagli anni '80 ad oggi

- ❑ MULTICS (OS timesharing);
- ❑ UNIX (versione ridotta e monoutente di MULTICS);
- ❑ Linux (OS *clone* di UNIX);
- ❑ Android è uno strato software "sopra" Linux.
- ❑ MacOS deriva da Berkeley UNIX;
- ❑ iOS è "simile" a macOS;

*Berkeley Software Distribution (BSD)



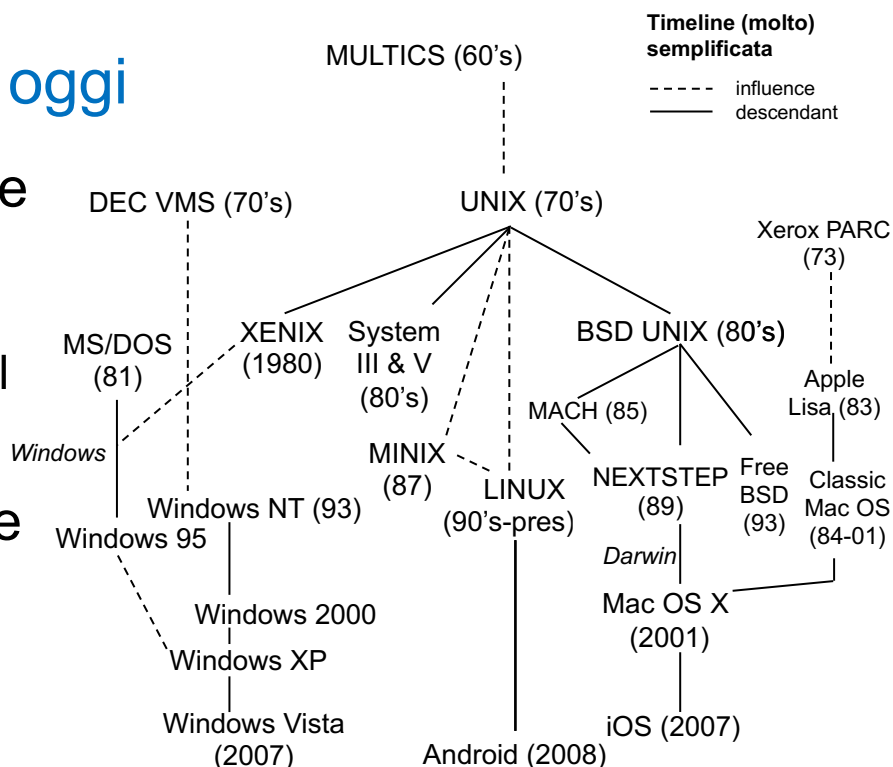
Dagli anni '80 ad oggi

❑ Quarta generazione (1980 ad oggi):

- large scale integration; personal computers.

❑ Quinta generazione (1990 ad oggi):

- computer mobili.



Domini applicativi OS

- ❑ Mainframe Operating Systems;
- ❑ Server Operating Systems;
- ❑ Personal Computer Operating Systems;
- ❑ Smartphone and Handheld Computer Operating Systems;
- ❑ Distributed Operating Systems;
- ❑ The Internet of Things (IoT) and Embedded Operating Systems;
- ❑ Real-Time Operating Systems;
- ❑ Smart Card Operating Systems.

STRUTTURA DI UN SO

Cap. 1: par. 1.7 (fino a 1.7.3 incluso)

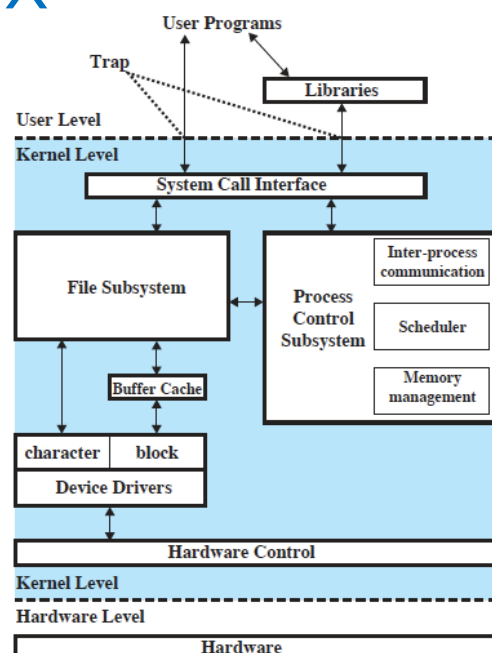
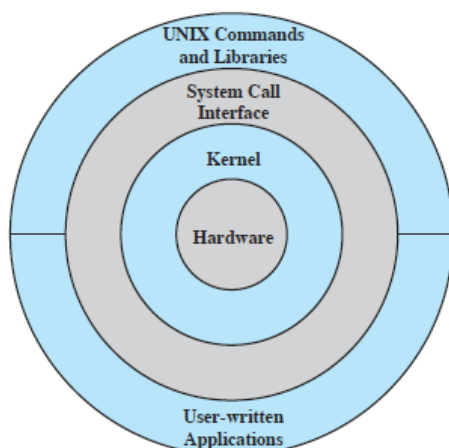
Struttura di un SO

- Un SO è un programma di notevole **complessità** e **dimensione**.
- È fondamentale applicare durante il suo progetto e realizzazione le tecniche proprie dell'ingegneria del software, al fine di garantire:
 - correttezza, modularità, facilità di manutenzione, efficienza di esecuzione, etc.

Sistema monolitico (macrokernel)

- L'intero SO come singolo programma in kernel mode:
 - **pro**: velocità di esecuzione;
 - **contro**: dimensioni del file binario, robustezza.
 - i "primi" SO monolitici erano effettivamente costituiti da un solo programma senza particolari suddivisioni;
- In pratica OS **monolitico** + **progettazione modulare**:
 - il SO è organizzato in moduli, ciascuno destinato ad offrire una delle funzionalità del sistema;
 - Esempi: kernel **Unix-like** (Linux, BSD, AIX, System V).

Architettura modulare: UNIX

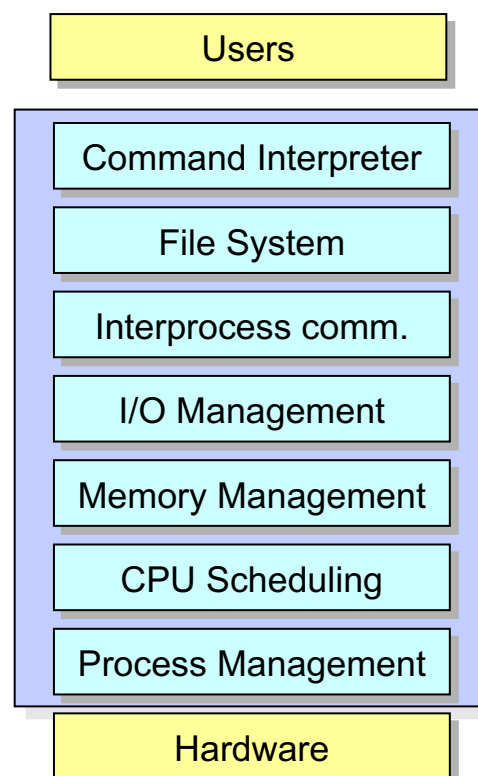


Moduli caricabili

- ❑ File oggetto contenente funzionalità del kernel non indispensabili per l'inizializzazione delle periferiche:
 - attivabile/disattivabile (on-demand) a tempo di esecuzione (a mano oppure automaticamente);
 - rappresentano un modo per ridurre la dimensione del kernel ed aumentare la robustezza.
- ❑ Esempio Linux:
 - `lsmod` elenca i moduli caricati;
 - `insmod/rmmod` consentono di caricare/rimuovere un modulo.

Sistema a livelli

- ❑ SO modulari, organizzati in una struttura gerarchica;
- ❑ Le interazioni hanno luogo solo tra livelli adiacenti;
- ❑ Esempio: **THE** (Technische Hogeschool Eindhoven).



Una riflessione

- Si ipotizza che per sistemi 'seri' ci possano essere da 2 a 10 difetti ogni 1000 LOC ...
- ... per 5 milioni LOC significa **10000-50000** difetti!

Microkernel

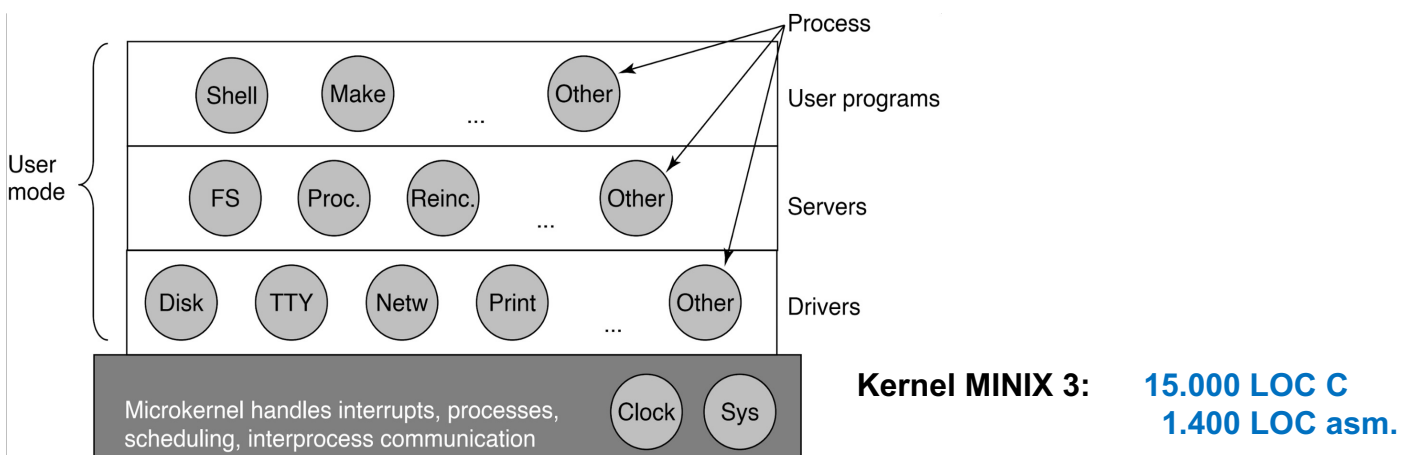


Figure 1-26. Simplified structure of the MINIX system

Microkernel

- Un **microkernel** implementa solo i meccanismi assolutamente necessari:
 - le restanti funzionalità sono implementate all'esterno del kernel, da processi di sistema (in **user mode**);
 - **pro**: facilità di estensione, robustezza ai guasti;
 - **contro**: performance, scambi di messaggi più lenti di una chiamata a funzione (come nei kernel monolitici);
 - Esempi: **Tru64 UNIX (Digital), Mach, Minix**.

Hybrid kernel (soluzione intermedia)

- Soluzione intermedia tra macro e microkernel:
 - i driver dei dispositivi eseguono in **kernel mode**;
 - Esempi: **Windows, OS X**.

Confronto

□ Schema esemplificativo:

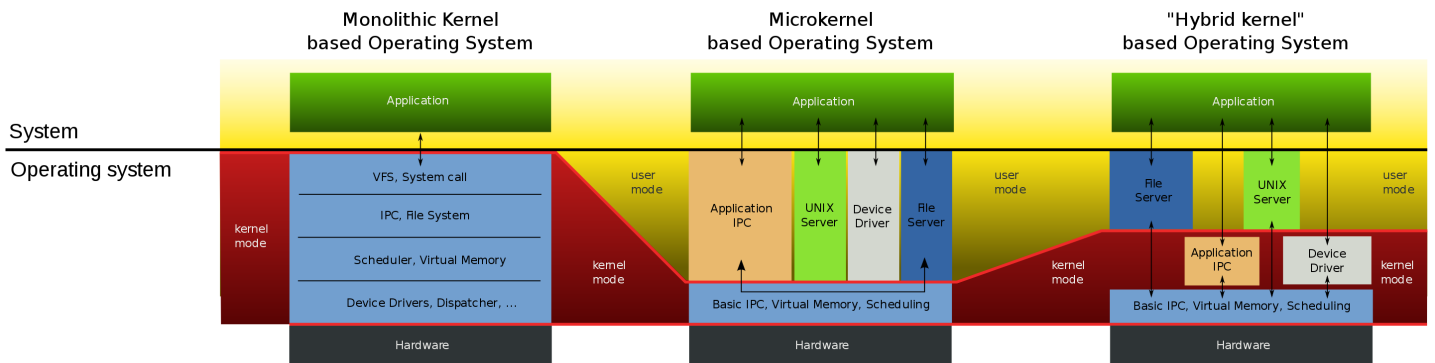


Image source: Wikipedia (<https://en.wikipedia.org/>)